PySpark DataFrame 101: Getting started with PySpark DataFrames

Archit Saxena · Follow
6 min read · Feb 27, 2023

PySpark Introduction

PySpark is a Python library that provides a Python API for Apache Spark and enables us to perform data processing and analysis at scale. PySpark DataFrames are a distributed collection of data organised into named columns, similar to a table in a relational database.

Source: https://www.linkedin.com/pulse/introduction-pyspark-what-isit-leonardo-anello

PySpark DataFrame

PySpark DataFrames are designed to handle large datasets that require distributed processing across multiple machines. They are optimised for scalability and can process data in parallel across a cluster of machines, making them well-suited for big data processing. PySpark also provides a range of built-in functions and APIs that enable us to perform complex data analysis and machine learning tasks on large datasets. This gives **PySpark an advantage over Pandas** when we are dealing with large datasets.

Lazy Evaluation in PySpark DataFrame

PySpark DataFrames use lazy evaluation, which means that data is not processed until an action is triggered.

Lazy evaluation is a technique where a program waits to do something until it absolutely has to. In PySpark, this means that operations on a DataFrame are not done right away. Instead, they are only done when they have to be.

This can save time and resources because it allows Spark to optimise the way it performs its operations.

For example, imagine we have a DatFrame with a million rows of data. If we want to filter that data, Spark won't do it right away. Instead, it will wait until it needs to.

PySpark DataFrame in-action

In this section, we will install, set up and learn about some PySpark DataFrame operations. We will be using Google Colab.

PySpark installation and set up

To install PySpark, we need to run the following command —

We need to create a spark session to interact with Apache Spark. To create, we need to run the following commands —

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('Spark101').getOrCreate()
spark
```

Fig: The output of the above code

Reading CSV and understanding data

Reading a CSV file is similar to that in Pandas.

We will use the CSV file from *this Kaggle dataset*. To read a CSV, we need the following command —

```
df_pyspark = spark.read.csv('Salary_Dataset_with_Extra_Features.csv', header=Tru
df_pyspark.show(5)
```

```
+------+--------------------+----------------+-------+----------------+---------+-----------------+---------+
|Rating|        Company Name|       Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+--------------------+----------------+-------+----------------+---------+-----------------+---------+
|   3.8|              Sasken|Android Developer| 400000|               3|Bangalore|        Full Time|  Android|
|   4.5|Advanced Millenni...|Android Developer| 400000|               3|Bangalore|        Full Time|  Android|
|   4.0|           Unacademy|Android Developer|1000000|               3|Bangalore|        Full Time|  Android|
|   3.8|  SnapBizz Cloudtech|Android Developer| 300000|               3|Bangalore|        Full Time|  Android|
|   4.4|Appoids Tech Solu...|Android Developer| 600000|               3|Bangalore|        Full Time|  Android|
+------+--------------------+----------------+-------+----------------+---------+-----------------+---------+
only showing top 5 rows
```

Dataset count

```
print('No. of rows in our dataset:', df_pyspark.count())
```

```
No. of rows in our dataset: 22770
```

Fig: DataFrame Count

Describing dataset

```
df_pyspark.describe().show()
```

```
+-------+------------------+--------------------+--------------------+------------------+------------------+------------------+---------+------------------+---------+
|summary|            Rating|        Company Name|           Job Title|            Salary| Salaries Reported| Location|Employment Status|Job Roles|
+-------+------------------+--------------------+--------------------+------------------+------------------+---------+------------------+---------+
|  count|             22770|               22770|               22770|             22770|             22770|    22770|             22770|    22770|
|   mean| 3.918212560386559|                 NaN|                null|695387.2112428634|1.8557751427316644|     null|              null|     null|
| stddev|0.5196753451613407|                 NaN|                null|884399.0136761835| 6.823668180058269|     null|              null|     null|
|    min|               1.0|(X,Y,Z) Architect...|      """ORACLE DBA"""|            100000|                 1|Bangalore|        Contractor|  Android|
|    max|               5.0|eNotice Ninja Pluss|          oracle dba|            996000|                98|     Pune|           Trainee|      Web|
+-------+------------------+--------------------+--------------------+------------------+------------------+---------+------------------+---------+
```

Fig: DataFrame describe

Dataset Schema

```
df_pyspark.printSchema()
```

```
root
 |-- Rating: string (nullable = true)
 |-- Company Name: string (nullable = true)
 |-- Job Title: string (nullable = true)
 |-- Salary: string (nullable = true)
 |-- Salaries Reported: string (nullable = true)
 |-- Location: string (nullable = true)
 |-- Employment Status: string (nullable = true)
 |-- Job Roles: string (nullable = true)
```

Fig: Column types

As we can see, all the columns are of `string` type. Although PySpark implicitly type casts the columns during comparison or other operations, it is always better to explicitly cast the column to be safe and certain about it.

Changing the column type

```
from pyspark.sql.types import IntegerType, FloatType
df_pyspark = df_pyspark.withColumn("Rating", df_pyspark["Rating"].cast(FloatType
df_pyspark = df_pyspark.withColumn("Salary", df_pyspark["Salary"].cast(IntegerTy
df_pyspark = df_pyspark.withColumn("Salaries Reported", df_pyspark["Salaries Rep

df_pyspark.printSchema()
```

```
root
 |-- Rating: float (nullable = true)
 |-- Company Name: string (nullable = true)
 |-- Job Title: string (nullable = true)
 |-- Salary: integer (nullable = true)
 |-- Salaries Reported: integer (nullable = true)
 |-- Location: string (nullable = true)
 |-- Employment Status: string (nullable = true)
 |-- Job Roles: string (nullable = true)
```

Fig: Column types

Selecting columns

We can select a **single column** to view —

```
df_pyspark.select('Salary').show(5)
```

```
+-------+
| Salary|
+-------+
| 400000|
| 400000|
|1000000|
| 300000|
| 600000|
+-------+
only showing top 5 rows
```

Fig: Top 5 rows of a single column

We can also select **multiple columns** —

```
df_pyspark.select(['Rating', 'Company Name', 'Job Title', 'Salary']).show(5)
```

```
+------+--------------------+-----------------+-------+
|Rating|        Company Name|        Job Title| Salary|
+------+--------------------+-----------------+-------+
|   3.8|              Sasken|Android Developer| 400000|
|   4.5|Advanced Millenni...|Android Developer| 400000|
|   4.0|           Unacademy|Android Developer|1000000|
|   3.8|  SnapBizz Cloudtech|Android Developer| 300000|
|   4.4|Appoids Tech Solu...|Android Developer| 600000|
+------+--------------------+-----------------+-------+
only showing top 5 rows
```

Fig: Top 5 rows of multiple columns

To get the **distinct values** of a column —

```
df_pyspark.select('Employment Status').distinct().show()
```

```
+-----------------+
|Employment Status|
+-----------------+
|       Contractor|
|           Intern|
|          Trainee|
|        Full Time|
+-----------------+
```

Fig: Distinct values of a column

Filtering columns

We can filter the DataFrame based on a **single column** —

```
df_pyspark.filter(df_pyspark['Salary'] > 1000000).show(5)
```

```
+------+------------------+-----------------+-------+-----------------+---------+-----------------+---------+
|Rating|      Company Name|        Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+------------------+-----------------+-------+-----------------+---------+-----------------+---------+
|   4.3|  Ashwin Infotrron|Android Developer|1188000|                1|Bangalore|        Full Time|  Android|
|   4.1|             Dunzo|Android Developer|2800000|                1|Bangalore|        Full Time|  Android|
|   4.0|      Grab (India)|Android Developer|1200000|                1|Bangalore|        Full Time|  Android|
|   4.0|Walmart Global Tech|Android Developer|2300000|                1|Bangalore|        Full Time|  Android|
|   4.0|           Deloitte|Android Developer|1200000|                1|Bangalore|        Full Time|  Android|
+------+------------------+-----------------+-------+-----------------+---------+-----------------+---------+
only showing top 5 rows
```

Fig: Top 5 rows filtered on a single column

Or **multiple columns —**

```
df_pyspark.filter((df_pyspark['Salary'] > 1000000) & (df_pyspark['Job Roles'] !=
```

```
+------+-------------+--------------------+-------+-----------------+---------+-----------------+---------+
|Rating| Company Name|           Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+-------------+--------------------+-------+-----------------+---------+-----------------+---------+
|   4.5|   Money View|   Backend Developer|1300000|                2|Bangalore|        Full Time|  Backend|
|   4.9|      Quizizz|   Backend Developer|2800000|                2|Bangalore|        Full Time|  Backend|
|   3.8|       Amazon|   Backend Developer|2800000|                1|Bangalore|        Full Time|  Backend|
|   4.1|          IBM|Backend Developer...|1600000|                1|Bangalore|       Contractor|  Backend|
|   4.3|Cisco Systems|   Backend Developer|1600000|                1|Bangalore|        Full Time|  Backend|
+------+-------------+--------------------+-------+-----------------+---------+-----------------+---------+
only showing top 5 rows
```

Fig: Top 5 rows filtered on multiple columns

`isin` filter can be applied on a column to filter **based on a list of strings —**

```
df_pyspark.filter(df_pyspark['Company Name'].isin('Unacademy', 'Amazon')).show(5
```

```
+------+------------+--------------------+-------+-----------------+---------+-----------------+---------+
|Rating|Company Name|           Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+------------+--------------------+-------+-----------------+---------+-----------------+---------+
|   4.0|   Unacademy|   Android Developer|1000000|                3|Bangalore|        Full Time|  Android|
|   3.8|      Amazon|   Android Developer| 228000|                1|Bangalore|        Full Time|  Android|
|   3.8|      Amazon|   Android Developer| 300000|                1|Bangalore|        Full Time|  Android|
|   3.8|      Amazon|Android Software ...| 624000|                1|Hyderabad|        Full Time|  Android|
|   3.8|      Amazon|Android Developer...| 984000|                1|New Delhi|           Intern|  Android|
+------+------------+--------------------+-------+-----------------+---------+-----------------+---------+
only showing top 5 rows
```

Fig: Top 5 rows filtered based on a list

Adding, deleting and renaming columns

We can add a custom column which helps us in feature engineering —

```
df_pyspark = df_pyspark.withColumn('Rating>=4', df_pyspark['Rating']>=4.0)
df_pyspark.show(5)
```

```
+------+--------------------+--------------------+-------+-----------------+---------+-----------------+---------+----------+
|Rating|        Company Name|           Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|Rating>=4|
+------+--------------------+--------------------+-------+-----------------+---------+-----------------+---------+----------+
|   3.8|              Sasken|Android Developer| 400000|                3|Bangalore|        Full Time|  Android|     false|
|   4.5|Advanced Millenni...|Android Developer| 400000|                3|Bangalore|        Full Time|  Android|      true|
|   4.0|           Unacademy|Android Developer|1000000|                3|Bangalore|        Full Time|  Android|      true|
|   3.8|  SnapBizz Cloudtech|Android Developer| 300000|                3|Bangalore|        Full Time|  Android|     false|
|   4.4|Appoids Tech Solu...|Android Developer| 600000|                3|Bangalore|        Full Time|  Android|      true|
+------+--------------------+--------------------+-------+-----------------+---------+-----------------+---------+----------+
only showing top 5 rows
```

Fig: Top 5 rows after adding a column

Generally, as a best practice column names should not contain special characters except underscore (_).

Hence we need to **rename** our column to remove the special characters —

```
df_pyspark = df_pyspark.withColumnRenamed('Rating>=4', 'RatingGrtThn4')
df_pyspark.show(5)
```

```
+------+--------------------+-----------------+-------+-----------------+---------+-----------------+---------+-------------+
|Rating|        Company Name|        Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|RatingGrtThn4|
+------+--------------------+-----------------+-------+-----------------+---------+-----------------+---------+-------------+
|   3.8|              Sasken|Android Developer| 400000|                3|Bangalore|        Full Time|  Android|        false|
|   4.5|Advanced Millenni...|Android Developer| 400000|                3|Bangalore|        Full Time|  Android|         true|
|   4.0|           Unacademy|Android Developer|1000000|                3|Bangalore|        Full Time|  Android|         true|
|   3.8|  SnapBizz Cloudtech|Android Developer| 300000|                3|Bangalore|        Full Time|  Android|        false|
|   4.4|Appoids Tech Solu...|Android Developer| 600000|                3|Bangalore|        Full Time|  Android|         true|
+------+--------------------+-----------------+-------+-----------------+---------+-----------------+---------+-------------+
only showing top 5 rows
```

Fig: Top 5 rows after renaming the column

Let's **delete** the column using `drop` function —

```
df_pyspark = df_pyspark.drop('RatingGrtThn4')
df_pyspark.show(5)
```

```
+------+--------------------+-----------------+-------+-----------------+---------+-----------------+---------+
|Rating|        Company Name|        Job Title| Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+--------------------+-----------------+-------+-----------------+---------+-----------------+---------+
|   3.8|              Sasken|Android Developer| 400000|                3|Bangalore|        Full Time|  Android|
|   4.5|Advanced Millenni...|Android Developer| 400000|                3|Bangalore|        Full Time|  Android|
|   4.0|           Unacademy|Android Developer|1000000|                3|Bangalore|        Full Time|  Android|
|   3.8|  SnapBizz Cloudtech|Android Developer| 300000|                3|Bangalore|        Full Time|  Android|
|   4.4|Appoids Tech Solu...|Android Developer| 600000|                3|Bangalore|        Full Time|  Android|
+------+--------------------+-----------------+-------+-----------------+---------+-----------------+---------+
only showing top 5 rows
```

Fig: Top 5 rows after dropping the column

Sorting columns

Sorting based on a **single column** —

```
df_pyspark.sort('Salary', ascending = False).show(5)
```

```
+------+----------------+--------------------+--------+----------------+---------+-----------------+---------+
|Rating|    Company Name|           Job Title|  Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+----------------+--------------------+--------+----------------+---------+-----------------+---------+
|   3.6|Thapar University|Software Developm...|90000000|               1|New Delhi|        Full Time|      SDE|
|   3.8|       Concentrix|Oracle Database A...|10000000|               1|Bangalore|        Full Time| Database|
|   3.6|OASYS Cybernetics|Senior Java Devel...|10000000|               1|  Chennai|        Full Time|     Java|
|   3.5|   Koru UX Design|Senior Front End ...|10000000|               1|     Pune|        Full Time| Frontend|
|   3.7|   Nityo Infotech|Lead UI Designer,...| 9900000|               1|Bangalore|        Full Time| Frontend|
+------+----------------+--------------------+--------+----------------+---------+-----------------+---------+
only showing top 5 rows
```

Fig: Top 5 sorted rows on a single column

Sorting based on **multiple columns**, with different orders —

```
df_pyspark.sort(df_pyspark['Salary'].desc(),df_pyspark['Rating'].asc()).show(5)
```

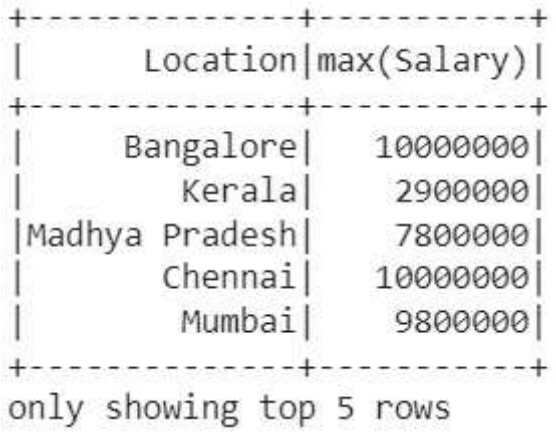
```
+------+----------------+--------------------+--------+----------------+---------+-----------------+---------+
|Rating|    Company Name|           Job Title|  Salary|Salaries Reported| Location|Employment Status|Job Roles|
+------+----------------+--------------------+--------+----------------+---------+-----------------+---------+
|   3.6|Thapar University|Software Developm...|90000000|               1|New Delhi|        Full Time|      SDE|
|   3.5|   Koru UX Design|Senior Front End ...|10000000|               1|     Pune|        Full Time| Frontend|
|   3.6|OASYS Cybernetics|Senior Java Devel...|10000000|               1|  Chennai|        Full Time|     Java|
|   3.8|       Concentrix|Oracle Database A...|10000000|               1|Bangalore|        Full Time| Database|
|   3.7|   Nityo Infotech|Lead UI Designer,...| 9900000|               1|Bangalore|        Full Time| Frontend|
+------+----------------+--------------------+--------+----------------+---------+-----------------+---------+
only showing top 5 rows
```

Fig: Top 5 sorted rows on multiple columns with different orders

GroupBy and Aggregate functions

Grouping by a **single column** —

```
df_pyspark.groupBy('Location').max('Salary').show(5)
```

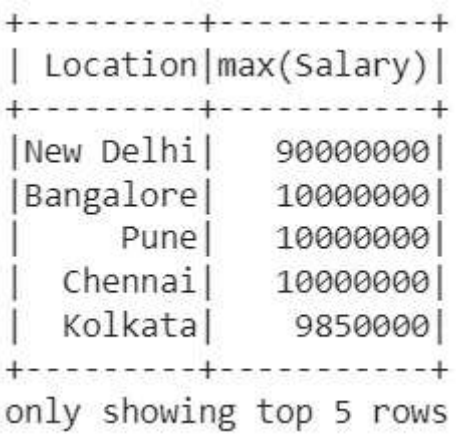


```
+--------------+-----------+
|      Location|max(Salary)|
+--------------+-----------+
|     Bangalore|   10000000|
|        Kerala|    2900000|
|Madhya Pradesh|    7800000|
|       Chennai|   10000000|
|        Mumbai|    9800000|
+--------------+-----------+
only showing top 5 rows
```

Fig: Max value of 'Salary' column, grouped by 'Location' column

Grouping and sorting by a single column —

```
df_pyspark.groupBy('Location').max('Salary').sort('max(Salary)', ascending = Fal
```



```
+---------+-----------+
| Location|max(Salary)|
+---------+-----------+
|New Delhi|   90000000|
|Bangalore|   10000000|
|     Pune|   10000000|
|  Chennai|   10000000|
|  Kolkata|    9850000|
+---------+-----------+
only showing top 5 rows
```

Fig: Sorted max value of 'Salary' column, grouped by 'Location' column

Grouping by **multiple columns** —

```
df_pyspark.groupBy('Location', 'Employment Status').max('Salary').show(5)
```

```
+---------+-----------------+-----------+
| Location|Employment Status|max(Salary)|
+---------+-----------------+-----------+
|Hyderabad|        Full Time|    9700000|
|Bangalore|          Trainee|     300000|
|     Pune|           Intern|    1800000|
|  Kolkata|        Full Time|    9850000|
|     Pune|       Contractor|    2900000|
+---------+-----------------+-----------+
only showing top 5 rows
```

Fig: Max value of 'Salary' column, grouped by 'Location' and 'Employment Status' columns

Grouping by a single column and taking averages of multiple columns—

```
df_pyspark.groupBy('Location').avg().select(['Location', 'avg(Rating)', 'avg(Sal
```

```
+--------------+------------------+-----------------+
|      Location|       avg(Rating)|      avg(Salary)|
+--------------+------------------+-----------------+
|     Bangalore|3.9202323313748386|735344.7395934173|
|        Kerala| 3.885185190924892|553577.4814814815|
|Madhya Pradesh| 3.992258064208492|677641.9096774193|
|       Chennai|3.9027257909902815|584559.6615134255|
|        Mumbai|3.8817089433186203|961180.3684913218|
+--------------+------------------+-----------------+
only showing top 5 rows
```

Fig: Average values of 'Rating' and 'Salary' columns, grouped by 'Location'

Grouping by a single column and **aggregating** on multiple columns —

```
df_pyspark.groupBy('Location').agg({"Rating": "avg", "Salary": "max"}).show(5)
```

```
+--------------+-----------+------------------+
|      Location|max(Salary)|       avg(Rating)|
+--------------+-----------+------------------+
|     Bangalore|   10000000|3.9202323313748386|
|        Kerala|    2900000| 3.885185190924892|
|Madhya Pradesh|    7800000| 3.992258064208492|
|       Chennai|   10000000|3.9027257909902815|
|        Mumbai|    9800000|3.8817089433186203|
+--------------+-----------+------------------+
only showing top 5 rows
```

Fig: Average values of ‘Rating’ and Max value of ‘Salary’ columns, grouped by ‘Location’

Combining multiple operations

We can also combine the operations mentioned above as per our requirements. One example is given below where I have grouped by, then took max, followed by filtering and finally sorting the result

```
df_pyspark.groupBy('Location', 'Employment Status').max('Salary').filter(df_pysp
```

```
+---------+-----------------+-----------+
| Location|Employment Status|max(Salary)|
+---------+-----------------+-----------+
|New Delhi|        Full Time|   90000000|
|New Delhi|      Contractor|    4800000|
|New Delhi|           Intern|    2000000|
|New Delhi|         Trainee|     120000|
+---------+-----------------+-----------+
```

Fig: Multiple operations on DataFrame

Advanced operations

There are also some advanced operations in PySpark —

- Window: allows to perform complex aggregations and transformations over groups of rows.
- Explode: transforms a single column with arrays or maps into multiple rows.
- Pivot: allows for rotation of rows into columns.
- User-Defined Functions (UDFs): enables users to define their functions and use them in PySpark.
- ML module: provides machine learning algorithms and tools for building and evaluating models.
- Approximate functions: such as approx_mean, enable approximate calculations over large datasets for faster processing.

As this article is intended for those just starting with PySpark DataFrames, we won't cover these operations in detail.

Conclusion

PySpark is a Python library that provides a user-friendly interface to Apache Spark for big data processing and analysis. PySpark DataFrames are distributed data collections optimized for scalability and parallel processing. They use lazy evaluation, delaying processing until necessary to optimize performance. PySpark provides various DataFrame operations, such as CSV reading, column selection, filtering, sorting, grouping, aggregation and some advanced operations. Overall, PySpark is a powerful tool for handling large-scale datasets.